



Experiment 05: Containerizing Applications using Docker

Experiment No.	05
Mapped CO	CO4
Aim	To containerize an application by writing a Dockerfile, building an image and running it as a container.
Tools / Software	Docker Engine; A sample web application such as Python Flask or Node.js; Docker Hub account optional; Code editor / terminal

Theory

A container is an isolated and runnable instance of an application packaged with its required dependencies. Containers share the host operating-system kernel and therefore start faster and consume fewer resources than traditional virtual machines. Docker is a commonly used platform for building, running and managing containers.

A Docker image is an immutable, layered template created from instructions written in a Dockerfile. Common Dockerfile instructions include FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. When an image is executed, it becomes a container. Images can be tagged and stored in a registry such as Docker Hub for sharing and deployment.

Procedure

1. Install Docker Engine and verify the installation using `docker --version`.
2. Create a project directory containing a sample web application file and a dependency file such as `requirements.txt`.
3. Write a Dockerfile defining the base image, working directory, dependency installation, port exposure and start command.
4. Build the Docker image using `docker build` and assign a meaningful tag such as `myapp:1.0`.
5. Run the image as a container using `docker run` and map the container port to a host port.
6. List running containers using `docker ps` and inspect container logs using `docker logs`.
7. Open the application in a browser or use `curl` to verify that it is reachable on the mapped port.
8. Optionally tag and push the image to Docker Hub for sharing.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Docker Container"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, run and publish commands

```
docker --version
docker build -t myapp:1.0 .
docker run -d -p 5000:5000 --name myapp myapp:1.0
docker ps
docker logs myapp
curl http://localhost:5000
docker tag myapp:1.0 <user>/myapp:1.0
docker push <user>/myapp:1.0
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker installation verification, application creation, image building, container execution and output validation. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Pushkar Shinde	58	B3	_____

1. Docker Installation and Version Output

```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ docker --version
Docker version 29.7.2, build a7dcaa6
```

2. Application Source and Dependency Files Output



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



Pushkar3232 / DevOps-APSIT

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

DevOps-APSIT Public

main 1 Branch 0 Tags

Go to file Add file Code

Pushkar3232 ci: add automated build and test pipeline 54c91cc · 1 hour ago 7 Commits

.github/workflows	ci: add automated build and test pipeline	1 hour ago
app	ci: add automated build and test pipeline	1 hour ago
docker	chore: initialize DevOps project structure	1 hour ago
k8s	chore: initialize DevOps project structure	1 hour ago
terraform	chore: initialize DevOps project structure	1 hour ago
tests	ci: add automated build and test pipeline	1 hour ago
.gitignore	chore: initialize DevOps project structure	1 hour ago
README.md	merge: resolve README conflict	1 hour ago

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published [Create a new release](#)

Packages

No packages published [Publish your first package](#)

3. Dockerfile Output

```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                NAMES
672a02564328   0b3a8bb96a4e        "python app.py"         About a minute ago    Up About a minute    0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp    devops-apsit
```

4. Docker Image Build Output

```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ docker images
IMAGE                ID                  DISK USAGE   CONTENT SIZE   EXTRA
devops-apsit:1.0     472a076df363       216MB        52.3MB        U
```

5. Container Run, Logs and Application Access Output

```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ docker logs devops-apsit
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [19/Aug/2026 12:30:21] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [19/Aug/2026 12:30:43] "GET / HTTP/1.1" 200 -
```

6. Docker Hub Push Output (Optional)



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



Paste screenshot/output showing docker tag and docker push output if the image is uploaded to Docker Hub.

Paste output / screenshot here

GitHub Repository / Lab Folder Link: <https://github.com/Pushkar3232/DevOps-APSIT>

Docker Hub Repository Link (Optional):

Final Observation Statement:

Write 2-3 lines summarizing the result of the experiment.

Expected Output

The Docker image should build successfully, the container should run in the background, the application should be reachable on the mapped host port, and docker ps should list the running container. If Docker Hub is used, the tagged image should be visible in the repository.

Conclusion

The application was containerized by writing a Dockerfile, building a Docker image and running it as an isolated portable container, demonstrating image creation, container execution, port mapping and basic container lifecycle management.

Viva / Review Questions

1. Differentiate a container from a virtual machine.
2. What is the difference between a Docker image and a Docker container?
3. Explain the purpose of EXPOSE in a Dockerfile and the -p flag in docker run.
4. Why are Docker image layers cached?

Marks: ____ / 10

Date: _____ **Signature of Faculty:** _____